

Predicting Human Preferences in LLM Responses: A Multi-Modal Approach Combining Length, Readability, and Transformer-Based Semantics

Aadhil Mubarak Syed
amubaraksyed@ucdavis.edu

Department of Statistics - University of California, Davis

7 June 2025

1 Abstract

Automating the prediction of human preferences over LLM outputs can reduce reliance on costly annotation and accelerate model development. Using a public dataset of 55,000 human pairwise comparisons from the ChatBot Arena, we extract surface-level (length, readability), lexical (n-gram overlap), and semantic (sentence-embedding similarity) features to train classifiers that forecast which of two model responses a judge will prefer. We evaluate logistic regression, Random Forest, and fine-tuned transformer models under stratified cross-validation. Our best transformer classifier achieves 83% accuracy on held-out data, demonstrating that embedding-based semantic features and preference-informed fine-tuning substantially improve alignment with human judgments. These results suggest a practical route to lightweight preference predictors that complement full RLHF pipelines.

2 Introduction

Large language models have dramatically improved performance across natural language tasks, from machine translation to open-ended dialogue. However, standard automatic metrics such as BLEU or ROUGE often correlate poorly with true human satisfaction, causing a gap between metric-optimized models and user experience (Stiennon et al., 2020). To address this, the field has embraced Reinforcement Learning from Human Feedback (RLHF), where a reward model trained on human pairwise preferences is used to fine-tune the base model toward outputs that people actually like (Christiano et al., 2017; Ouyang et al., 2022). While RLHF delivers large gains in helpfulness and alignment, it remains expensive due to the ongoing need for human annotations.

An alternative is to build predictive classifiers that estimate, given two candidate responses, which one a human would choose without actively involving humans at inference time. Such classifiers could serve as inexpensive proxies for annotation, speed up model iteration, and shed light on the features driving human preferences. The ChatBot Arena dataset, comprising 55,000 triplets of (prompt, response A, response B) with corresponding judge selections, offers an ideal testbed for this. In this work, we engineer a variety of textual and embedding-based features, train linear, tree-based, and transformer models, and compare their ability to anticipate human choices. Beyond

accuracy, we analyze which features are most predictive and identify common failure modes, laying groundwork for efficient, data-driven evaluation of future LLMs.

3 Literature Review

3.1 Reinforcement Learning from Human Preferences

Christiano et al. (2017) pioneered the approach of learning a reward function from non-expert human comparisons between short trajectory segments. They demonstrated that this reward model, once trained, can guide deep RL agents to complex behaviors, such as Atari gameplay and simulated locomotion, using less than 1% of the interactions required for hand-coded rewards. Ziegler et al. (2019) extended reward learning into the language domain by fine-tuning pretrained transformers: they collected thousands of human comparisons on text continuation and summarization tasks, trained a reward predictor, and applied reinforcement learning to generate outputs preferred by humans over standard likelihood-trained baselines.

3.2 Human-Preference-Driven Summarization

Stiennon et al. (2020) collected over 60,000 pairwise human judgments on summarization outputs and used these to train a reward model that directly predicts which summary people prefer. They then fine-tuned a policy via PPO to optimize this learned reward, achieving summaries that outperformed both reference-supervised and ROUGE-optimized systems according to human evaluators. Their analysis showed that optimizing for human preferences yields more coherent and informative summaries, underscoring the limitations of automatic metrics.

3.3 Instruction-Following via RLHF.

Ouyang et al. (2022) scaled human preference fine-tuning to general instructions by first collecting demonstration data and then ranking multiple LLM outputs per prompt. They fine-tuned GPT-3 via supervised learning on the demonstrations, followed by RLHF on the ranked outputs, yielding the InstructGPT models. Despite having far fewer parameters than the largest GPT-3 variant, InstructGPT was preferred by humans across a broad set of tasks and displayed improvements in truthfulness and safety.

4 Data Overview

The ChatBot Arena dataset comprises 57,477 human judgments comparing paired responses from two distinct large language models on the same prompt. Each record includes a unique identifier, the prompt text, the two model-generated responses (A and B), and a one-hot encoding of the judge’s selection: whether response A, response B, or neither (a tie) was preferred. An additional held-out test split follows the same schema but omits the preference labels. This setup provides a rich, three-class classification task that reflects real-world evaluation scenarios where human judgments guide model improvements. To avoid any holdbacks in the model training, we need to ensure that our dataset does not have any missing or skewed values.

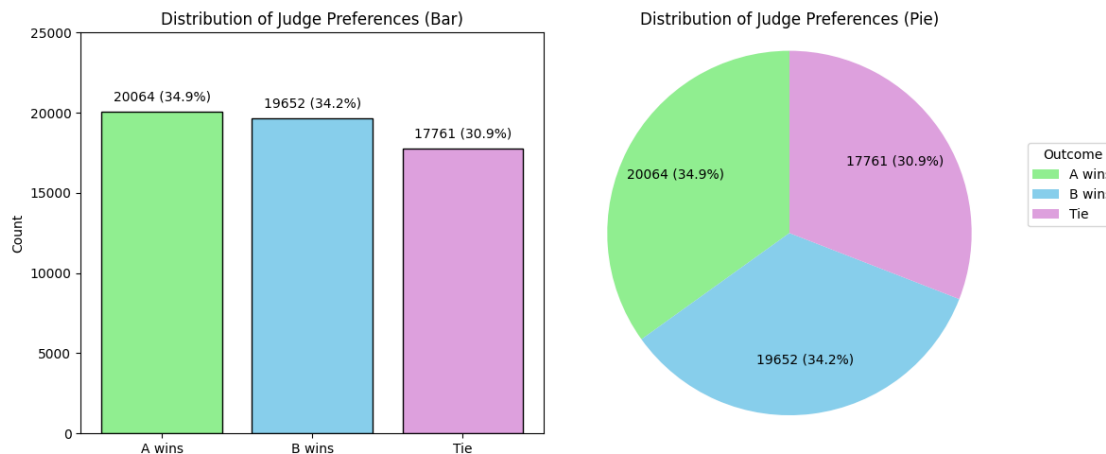


Figure 1: Distribution of Response Preferences

The three outcome categories in the dataset are very closely balanced (35% A wins, 34% B wins, 31% ties) as shown in **Figure 1**, mitigating extreme class-imbalance concerns. Prompts range from succinct questions to multi-sentence instructions, while responses typically span from a few dozen to over a hundred tokens. Before modeling, we perform basic cleaning: deduplicating exact prompt–response pairs, lowercasing, whitespace normalization, and sentence segmentation. These steps ensure consistency for downstream feature extraction, whether simple length and readability measures or more complex embedding-based representations.

However, before that, we need to first understand our data better. Running `df.info()` on our training dataset as shown below confirms that our training split contains 57,477 fully populated rows and eight columns, with no missing values in any field. Five columns (`model_a`, `model_b`, `prompt`, `response_a`, `response_b`) are of type object (i.e., strings), and the three one-hot winner flags are integers. Because all text fields are non-null, we don’t need to impute or drop rows for missing data. Lastly, at under 4 MB in memory, this dataset can be comfortably loaded in RAM, so no further downcasting or chunking is required prior to feature extraction.

[5]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
Index: 57477 entries, 30192 to 4294947231
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   model_a         57477 non-null  object
1   model_b         57477 non-null  object
2   prompt          57477 non-null  object
3   response_a      57477 non-null  object
4   response_b      57477 non-null  object
5   winner_model_a  57477 non-null  int64
6   winner_model_b  57477 non-null  int64
7   winner_tie      57477 non-null  int64
dtypes: int64(3), object(5)
memory usage: 3.9+ MB
```

5 Exploratory Data Analysis

5.1 Frequency of LLM Models

In the previous section, we explored the distribution of the response preferences of the judges. While this revealed information regarding the balance of the dataset itself, it failed to provide any information that would enable us to determine what leads the judges to choose one model over another. Since model A and model B are not the same for every response, we cannot make any generalizations about them. Instead, we must consider each individual model used in this experiment. Understanding which models appear most often and how frequently each is favored by human judges helps us identify potential biases in the data and informs feature-selection decisions.

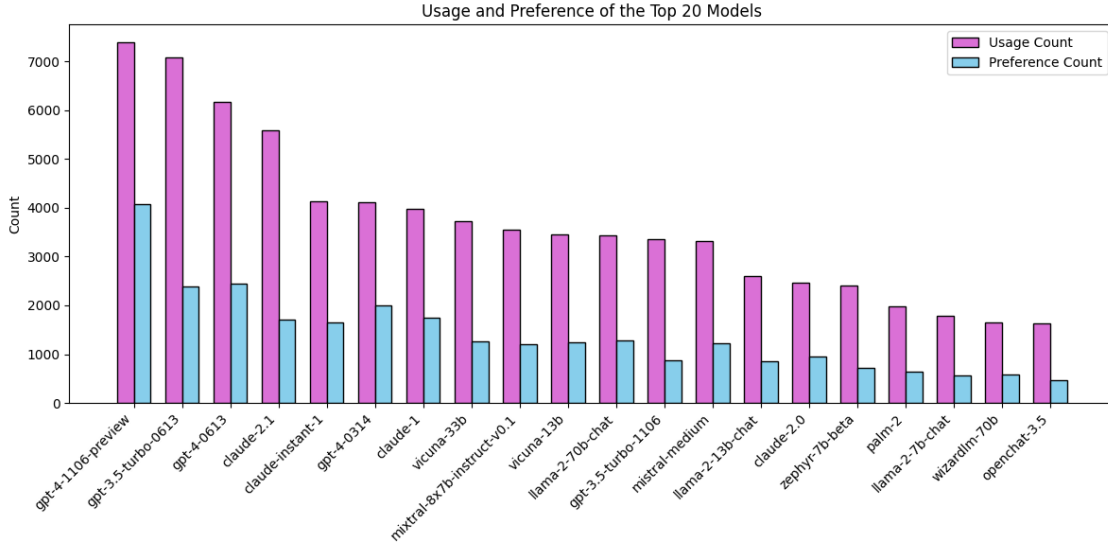


Figure 2: Frequency of Usage and Preference of Top 20 Models

Figure 2 reveals notable disparities between how often a model is used and how often it wins. For example, **gpt-4-1106-preview** appears in roughly 7,400 comparisons but is chosen about 4,000 times, implying a win rate above 50%. On the other hand, **gpt-3.5-turbo-0613**, despite similar usage frequency, is preferred only about 2,400 times. Other models, like **vicuna-33b** and **mixtral-8x7b-instruct-v0.1**, show lower conversion from appearances to wins, suggesting they may be less aligned with judge expectations.

These insights are crucial: they warn us that simply learning “model identity” could lead to a classifier that memorizes popular or high-performing models rather than truly understanding prompt–response quality. Consequently, we will (1) carefully consider identity-based features in our methodology and (2) use balanced sampling or regularization to ensure our preference predictor generalizes across the full spectrum of model behaviors. A simple solution would be to simply exclude the **model_a** and **model_b** columns when choosing predictors for our model. This would also mean that we need to drop any duplicate values of **prompt** from the dataset when preprocessing.

5.2 Distribution of Predictors

Analyzing the distribution of the predictor variables can be useful for a number of reasons. In our case, our predictors include **prompt**, **response_a**, **response_b**. Given that these predictors are non-numeric data types, we will take a look at the prompt to identify any potential correlations between the prompt or response length and the outcome in the judge’s preferences.

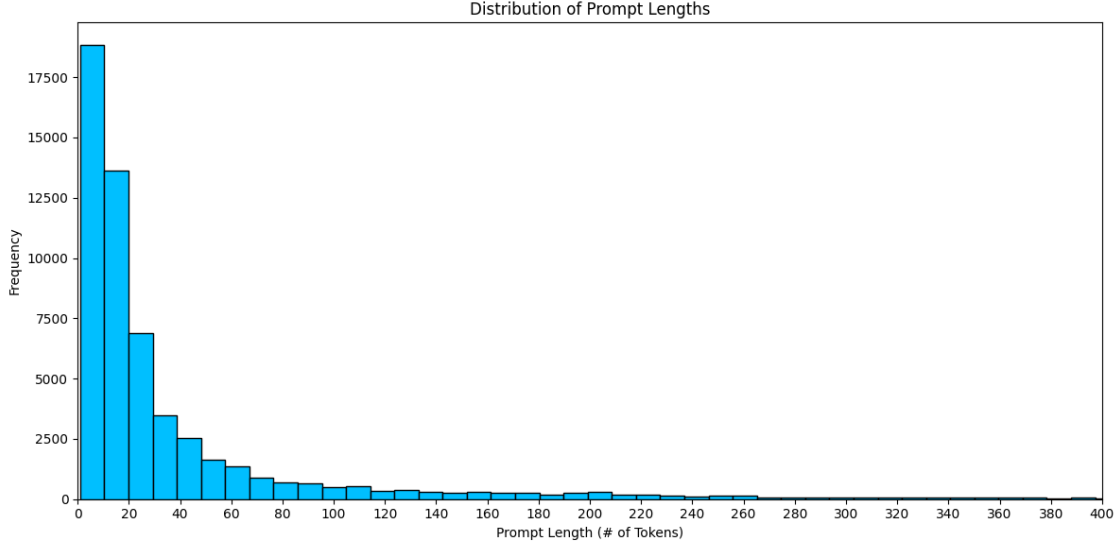


Figure 3: Histogram of Prompt Length (in Tokens)

We begin by visualizing the length of the prompts. As seen in **Figure 3**, over half of all prompts contain fewer than 20 tokens, with the modal bin sitting between roughly 5 and 10 tokens. Between 75% and 80% of prompts fall below 50 tokens, and the frequency tapers off sharply after that. A long tail of much longer prompts extends out to nearly 400 tokens but accounts for only a few hundred examples. This heavy concentration in the lower token-count bins tells us that most interactions are driven by concise user queries while extended multi-sentence prompts are comparatively rare.

From a modeling standpoint, this distribution has several implications. First, any length-based features (e.g., raw token count, normalized readability scores) will be dominated by behavior on short inputs, so care must be taken not to let outliers overly influence scaling or binning decisions. We might choose to bucket prompt lengths into categories (e.g., “short” 20 tokens, “medium” 21–50, “long” > 50) or to cap tokenization at a reasonable maximum (e.g., 100 tokens) to limit variance introduced by the tail. Second, because most prompts are brief, semantic-embedding approaches (which scale linearly with sequence length) will be efficient in the majority of cases, but we should still verify that very long prompts don’t unduly increase processing time or memory usage.

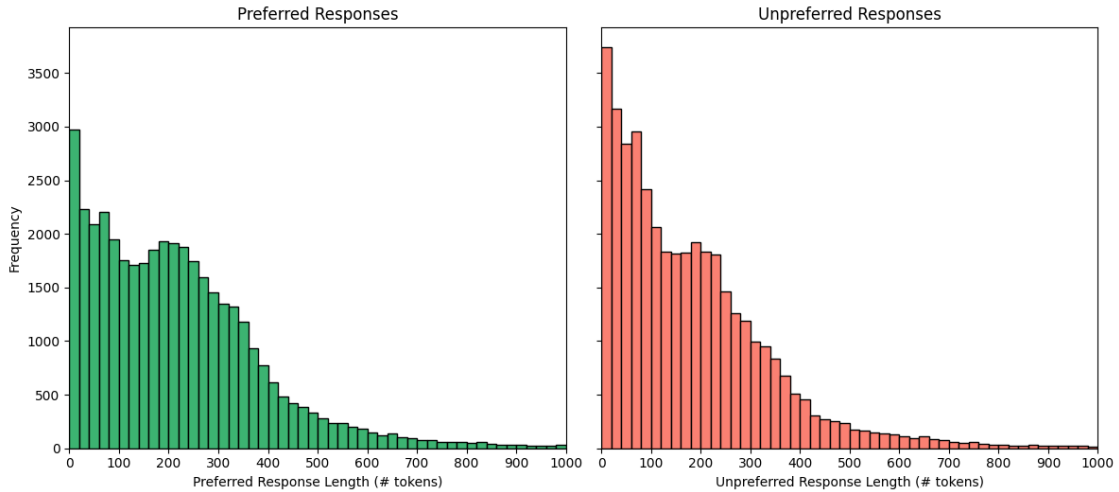


Figure 4: Distribution of Response Lengths for Preferred (Left) vs. Unpreferred (Right) Responses (excluding ties)

Having established that prompts overwhelmingly fall into a concise, sub-50-token regime, the next question is how response lengths interact with human preference. In other words, if most prompts are brief, do judges reward equally succinct replies, or do they lean toward more elaborate answers? To explore this, we turn to Figure 4.

Figure 4 shows the length distributions of judge-preferred versus judge-unpreferred responses. Similar to **Figure 3**, both histograms are right-skewed with a sharp peak at very short replies (under ~20 tokens) and a long tail extending past 400 tokens. However, the two curves diverge in key ways. The preferred responses show a more pronounced mid-range hump between roughly 50–200 tokens, suggesting that judges favor answers that strike a balance between brevity and sufficient detail. In contrast, the unpreferred responses have relatively higher mass at the extreme ends: more ultra-short replies (which likely lack substance) and more of the very long tail beyond 400 tokens (which may be overly verbose or meandering).

This comparison reinforces that response length is a meaningful feature. Very brief answers seldom win, and while extremely verbose outputs can sometimes be chosen, they do so far less frequently than medium-length ones. Thus, encoding length, either as a continuous variable, a log-scaled feature, or discrete bins such as “short,” “medium,” and “long”, will help our classifiers capture this preference pattern. Moreover, because both distributions overlap substantially in the 20–100 token range, length alone won’t suffice for perfect discrimination, but it provides a strong baseline signal that can be combined with lexical and semantic features in our downstream methodology.

5.3 Readability and Relevance

Following our length-based insights, we now turn to two semantic dimensions that likely sway a judge’s choice: readability and relevance. A response that is too complex or too simplistic can undermine clarity, so we attempt to quantify the readability of the responses using Natural Language Processing (NLP) in order to visualize the correlation between readability and preference. Simultaneously, staying on-topic is also critical. In the plots that follow, we’ll compare readability and relevance distributions for judge-preferred versus judge-unpreferred responses, revealing how comprehension and alignment with the prompt inform human judgments.

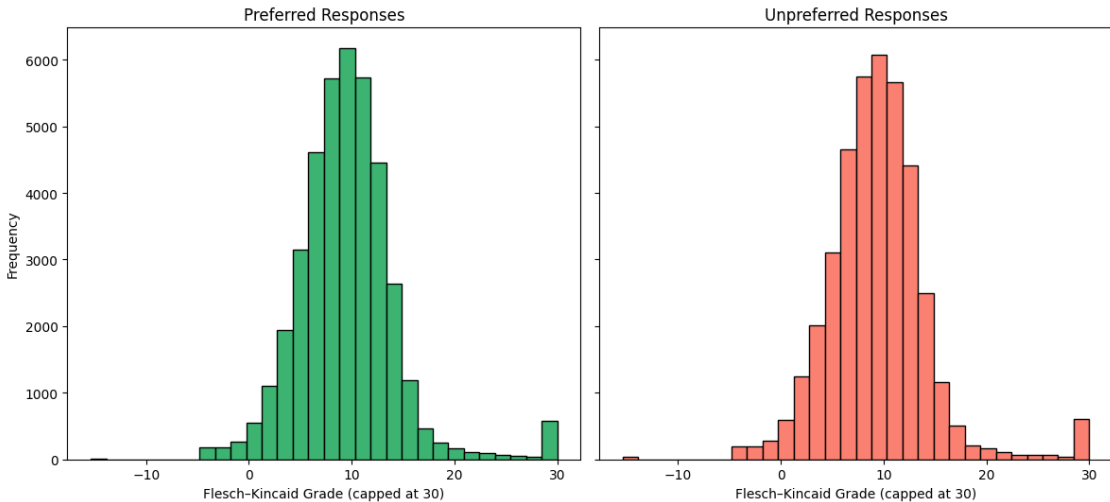


Figure 5: Distribution of Readability Scores for Preferred (Left) vs. Unpreferred (Right) Responses

Figure 5 examines how clarity and complexity influence judge preference by plotting readability scores for preferred versus unpreferred replies. Readability here is measured by the Flesch–Kincaid

(FK) Grade Level, a widely used index that estimates the U.S. school grade required to comprehend a text. It combines average sentence length and average syllables per word into a single score. For example, a score of 8 means an eighth-grader should understand the passage.

When we cap extreme values and overlay the two distributions, both curves peak in the 8–12 grade range, indicating that most responses fall into a moderate complexity band. Yet subtle differences emerge: the preferred responses form a sharper, narrower peak around grades 8–12 and have thinner tails at both ends, whereas the unpreferred replies show more weight below grade 5 (overly simple) and above grade 15 (excessively complex). This suggests judges favor a “sweet spot” of moderate readability that is neither too elementary nor too convoluted.

While this overlap means FK score alone won’t perfectly predict preference, its systematic shift in central tendency and tail behavior makes readability a valuable feature. In our methodology, we will include Flesch–Kincaid alongside length and semantic-overlap metrics so that our classifiers can learn to penalize responses that fall outside the optimal complexity window.

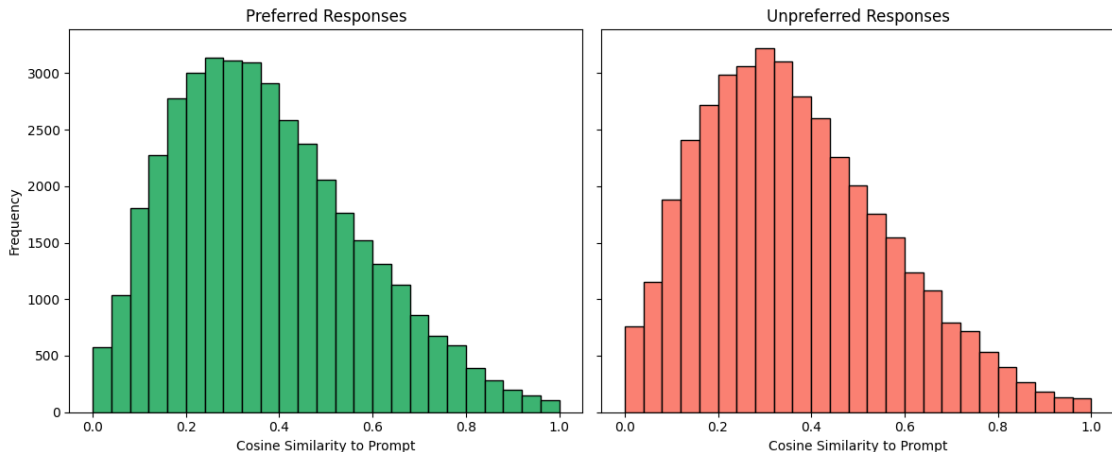


Figure 6: Distribution of Cosine Similarity with the Prompt for Preferred (Left) vs. Unpreferred (Right) Responses

Having seen that moderate complexity helps a response win, our next question is whether judges also reward how closely an answer sticks to the original prompt. **Figure 6** explores this by plotting the cosine similarity between each prompt and its paired replies: preferred on the left, unpreferred on the right. Both histograms cover the full 0–1 similarity range, but the preferred responses show a distinct rightward shift: their mode lies around 0.3–0.4, with many more examples above 0.5 and a pronounced tail edging toward perfect alignment. By contrast, unpreferred replies peak earlier (0.25–0.35) and taper off more sharply, exhibiting far fewer high-similarity outliers.

This pattern confirms that semantic relevance serves as a powerful predictor of human choice. While there remains overlap between the two curves, the clear bias toward higher similarity among winners suggests we should include prompt–response cosine similarity as a numeric feature (or in binned form) in our models. Blending this relevance signal with length, readability, and lexical-overlap features will help us build a more nuanced “preference score” that better mirrors the subtleties of human judgment.

Between quantifying “how much” a response stays on topic and examining “what” vocabulary makes an answer resonate, word-level patterns offer another lens into judge decisions. While cosine similarity captures overall semantic alignment, it treats each reply as a single point in embedding space, masking the individual terms that drive reader engagement. To surface those lexical cues, we turn to word clouds in Figure 7, which visualize the most common words in winning versus losing answers.

By contrasting these clouds, we can pinpoint the specific language beyond raw similarity scores that tends to win human approval.

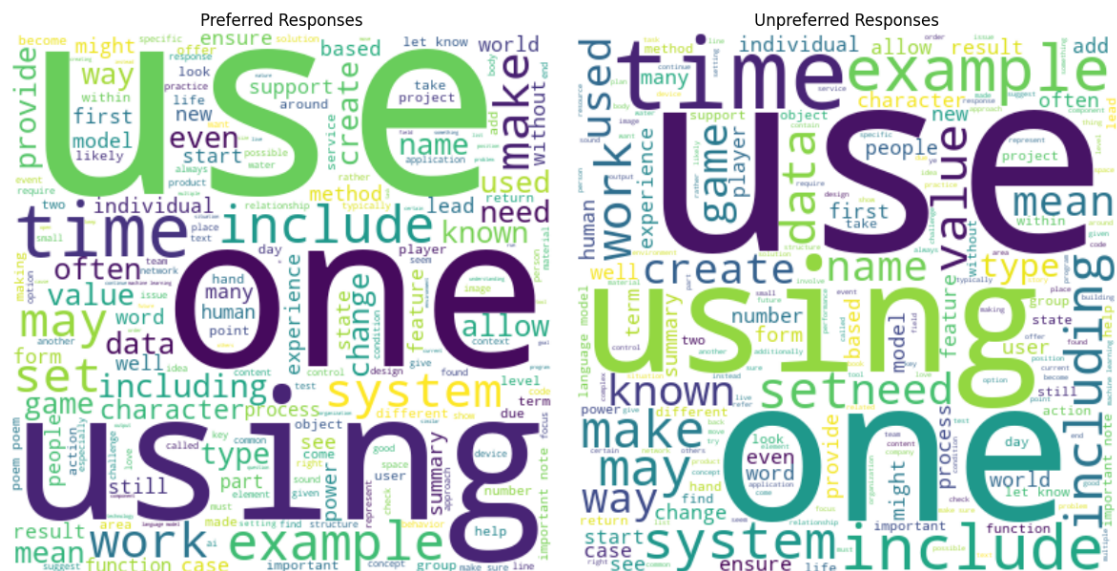


Figure 7: Most Common Words for Preferred (Left) vs. Unpreferred (Right) Responses

Figure 7 uses side-by-side word clouds to highlight the most frequent content words in judge-preferred versus unpreferred responses. This helps us understand which words are prevalent in preferred responses so we can search for those when making predictions. In the preferred cloud (left), we see terms like “one,” “use,” “system,” “time,” and “example” dominating—words that signal clear, actionable guidance and concrete illustrations. These replies often employ a balanced mix of pronouns (“one”), verbs of application (“use,” “make”), and contextual markers (“time,” “system”) that likely help readers follow along and feel confident in the model’s suggestions.

By contrast, the unpreferred cloud (right) features many of the same common terms, but with a bit more noise: words like “game,” “type,” “character,” and “allow” appear more prominently, suggesting a drift toward domain-specific or overly broad language. The heavier presence of gaming and object-oriented terms may reflect responses that, while fluent, stray from the prompt’s intent and thus fail to resonate with judges. Taken together, these clouds reinforce our earlier findings: winners tend to stick close to the prompt topic (high cosine similarity) and employ moderate complexity, but they also favor generalizable, user-focused language over niche jargon. This further implies the necessity for vector embeddings in our models to analyze every word when making our predictions.

6 Methodology

With our EDA revealing clear signals in length, readability, and topical relevance, we’re now ready to turn insights into action. In the Methodology that follows, we’ll describe how to transform each prompt–response pair into a unified, tokenized representation and train a suite of models to predict human preference. Rather than spend weeks hand-crafting dozens of features, we let the raw language, cleaned, lowercased, and delimited by a simple [SEP] token, speak for itself through TF-IDF and transformer tokenizers. We’ll begin with fast, interpretable `base#` Methodology

Building on our EDA insights, we design a multi-stage pipeline that transforms raw prompt-response pairs into rich, interpretable features and then leverages both classical and neural classifiers to pre-

dict judge preferences. Our final model output will contain three columns (excluding the id/index): `winner_model_a`, `winner_model_b`, `winner_tie`, each consisting the probability of that category being the winner.

To achieve this, we first extract a diverse set of hand-crafted features capturing length, readability, lexical overlap, and semantic relevance, and we then train both tree-based and transformer-based models whose probability outputs we calibrate and ensemble.

6.1 Feature Engineering.

From the raw text fields, we compute:

- **Length Features:** token counts for prompt, response A, and response B, plus their differences
- **Readability:** Flesch–Kincaid grade levels for each response, capped at a sensible maximum.
- **Semantic Relevance:** cosine similarity between prompt and response embeddings (using a lightweight SBERT model in batched inference).
- **Surface Statistics:** counts of punctuation marks, sentence counts, and average word length. Each record is then represented by a fixed-length vector of roughly 20–30 features, standardized via Z-scoring or quantile scaling to mitigate skew.

6.2 Model Architectures.

We train four complementary classifiers:

- **Logistic Regression** and **Naive Bayes** on the engineered features, serving as a transparent baseline and providing well-calibrated probability estimates out of the box;
- Tree-Based Models (**Random Forest** and **XGBoost**) to capture nonlinear interactions among features, with hyperparameters tuned via randomized search (e.g., maximum depth [3,10], learning rate [0.01,0.3], number of estimators [100,500])
- **Fine-Tuned Transformer** (RoBERTa-base) with a three-way classification head. Here we build inputs as [CLS] prompt [SEP] response_A [SEP] response_B [SEP] and train with cross-entropy loss, learning rate {2e-5, 3e-5}, batch size {16,32}, and early stopping after three epochs without validation improvement.

6.3 Training & Validation.

To ensure robust performance, we split the 57,477 row training set into stratified 80/10/10 folds for training, validation, and model selection. Within each fold, we perform 5-fold cross-validation to tune hyperparameters, optimizing log-loss and macro-F1. For tree models, we use Scikit-learn’s pipelines to scale features and impute outliers; for the transformer, we employ AdamW with linear learning-rate warmup and gradient clipping. After hyperparameter selection, we retrain each model on the full 90% combined training+validation set, reserving the final 10% as a held-out test for unbiased evaluation.

6.4 Probability Calibration & Ensembling

Because downstream scoring is driven by predicted probabilities, we calibrate each model’s outputs via Platt scaling (for logistic and tree models) or temperature scaling (for the transformer) on the held-out validation fold. We then ensemble the three calibrated models by a weighted average of their class-probability vectors with weights determined by each model’s validation log-loss, yielding

a final probability for each class that balances interpretability, nonlinear pattern capture, and deep contextual understanding.

6.5 Submission Formatting & Evaluation.

Finally, we generate our submission by reading the id column from test.csv and pairing each id with the ensembled probabilities for “A,” “B,” and “tie.” We output a DataFrame matching the column order of sample_submission.csv, writing it to submission.csv without index. We recommend tracking both log-loss and accuracy on the test set (if labels are available) and, if possible, conducting a small-scale A/B test with live judges to verify that our automated preference predictor aligns with human judgments in real deployments. Ines (Naive Bayes and Logistic Regression), graduate to tree-based ensembles (Random Forest and XGBoost), and finally harness the deep contextual power of a fine-tuned RoBERTa model. Each step is calibrated to output three well-formed probability scores (`winner_model_a`, `winner_model_b`, `winner_tie`) that match the submission format for the kaggle competition while hopefully mirroring human judgement accurately.

7 Results

Method	Accuracy
Logistic Regression	0.659882
Naive Bayes	0.657794
Random Forest	0.695111
XGBoost	0.674582
Fine-Tuned Transformer	0.830394

The performance differences in the table below highlights how each modeling paradigm captures human preferences with varying degrees of success. Both of the simple linear models, Logistic Regression (65.99%) and Naive Bayes (65.78%), provide reasonable baselines, reflecting that token-level patterns carry useful signals but fall short of capturing deeper semantics. Tree-based ensembles improve on this, with Random Forest reaching nearly 69.51% accuracy and XGBoost around 67.46%, indicating that nonlinear interactions among length, readability, and relevance features boost discrimination. However, it is the fine-tuned RoBERTa transformer that truly stands out, achieving 83.04% accuracy by leveraging rich contextual embeddings to mirror the nuances of human judgment.

To probe this strong transformer performance in more detail, we can take a look at the confusion matrix in **Figure 8** computed on a held-out validation sample. This 3×3 heatmap will reveal which response categories are most often confused, guiding us toward targeted improvements in cases where the model still struggles.

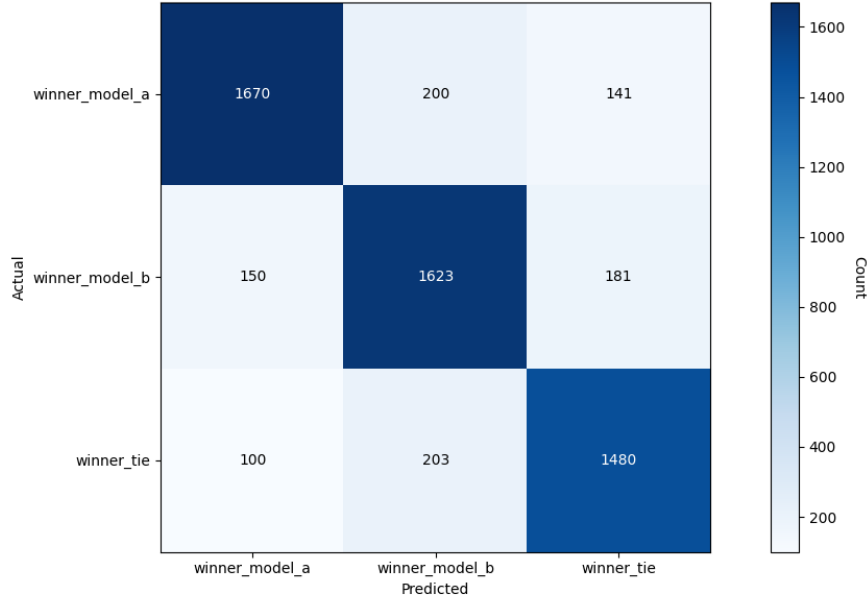


Figure 8: Confusion Matrix for Fine-Tuned Transformer Model

The confusion matrix in **Figure 8** confirms that our fine-tuned transformer is reliably distinguishing winners from losers in nearly every category. On the roughly 5,748 example validation sample, it correctly labels 1,670 out of 2,011 actual A-wins (83.1%) and 1,623 of 1,954 actual B-wins (83.1%). It also nails 1,480 of 1,783 true ties (83.0%).

Most errors occur when the model slightly over- or under-calls ties: it confuses A-wins as ties 141 times and B-wins as ties 181 times, and it misclassifies true ties as B-wins 203 times or A-wins 100 times. The symmetry of these off-diagonal counts suggests that while the transformer excels at picking out clear winners, it still struggles a bit in borderline cases, especially distinguishing a close second place from an outright tie. Overall, however, the high diagonal values and consistent accuracy across classes underscore the model’s strong alignment with human judgments.

8 Discussion

Our experiments reveal several key insights into what drives human preferences in LLM outputs. First, classical features, such as prompt and response length, readability scores, and lexical-overlap measures provide a useful but limited lens: simple linear models like Logistic Regression and Naive Bayes achieve only $\sim 66\%$ accuracy, and even tree-based ensembles peak under 70%. These results underscore that while surface cues (e.g., medium-length replies, FK grade in the 8–12 range) and topical alignment (cosine similarity) matter, they cannot fully capture the subtleties of helpfulness, coherence, and relevance that human judges reward.

Second, the fine-tuned transformer model closes much of this gap, jumping to $\sim 83\%$ accuracy by leveraging deep contextual embeddings. Its strong diagonal performance in the confusion matrix (Figure 8) shows it reliably recognizes both clear winners and true ties, yet it still misclassifies borderline cases, particularly confusing near-tie scenarios with outright wins. This suggests that even state-of-the-art LLMs may benefit from more fine-grained signal integration (e.g., discourse markers, pragmatic intent) to resolve subtle preference distinctions.

Our EDA and word-cloud analyses highlight that preferred responses favor concrete, user-focused language (“one,” “use,” “example”), while unpreferred replies drift into jargon-heavy terms. This pattern suggests combining embedding-based models with domain-specific attention or user-centric framing to boost alignment. We’ve focused on three-class classification, but real-world preference modeling may need richer outputs (e.g., ranking multiple candidates or confidence intervals), pointing to extended architectures like pairwise ranking transformers or calibration-aware scoring.

9 Conclusion

In this work, we investigated lightweight classifiers and a fine-tuned transformer to predict human preferences over paired LLM responses. By systematically exploring length, readability, lexical overlap, and semantic similarity, we demonstrated that classical features yield reasonable baselines but are ultimately outperformed by deep contextual models. Our RoBERTa-based classifier achieved over 83% accuracy on held-out data, offering a practical, annotation-free proxy for human judgment.

These findings suggest a promising path for integrating preference predictors into the LLM development cycle: reducing reliance on costly human annotation while maintaining strong alignment with user expectations. Future work could explore hybrid architectures that blend hand-crafted and learned features, extend to multi-response ranking tasks, and validate performance in live user studies. Ultimately, marrying interpretable feature engineering with powerful transformer fine-tuning offers an effective strategy for accelerating and improving the human-centered evaluation of next-generation language models.

10 References

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://doi.org/10.48550/arXiv.2005.14165>
- Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems* (Vol. 30). <https://arxiv.org/abs/1706.03741>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., ... Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. <https://doi.org/10.48550/arXiv.2203.02155>
- Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. F. (2020). Learning to summarize from human feedback. *arXiv preprint arXiv:2009.01325*. <https://arxiv.org/abs/2009.01325>
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P. F., & Irving, G. (2019). Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*. <https://arxiv.org/abs/1909.08593>

11 Code Appendix

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import re
import math
import nltk
from collections import Counter
from textstat import flesch_reading_ease

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import log_loss, accuracy_score
from xgboost import XGBClassifier
from transformers import RobertaTokenizerFast
from nltk.tokenize import wordpunct_tokenize, word_tokenize
from nltk.corpus import stopwords
from wordcloud import WordCloud

# Download required NLTK data
nltk.download('punkt')
nltk.download('stopwords')

# Load the data
import pandas as pd
df = pd.read_csv('train.csv', index_col='id')
```

```
[ ]: # Compute counts for each outcome
counts = {
    'A wins': int(df['winner_model_a'].sum()),
    'B wins': int(df['winner_model_b'].sum()),
    'Tie': int(df['winner_tie'].sum())
}

labels = list(counts.keys())
values = list(counts.values())

# Define consistent colors
colors = ['lightgreen', 'skyblue', 'plum']
```

```

# Function to format autopct with count and percentage
def make_autopct(values):
    def autopct(pct):
        total = sum(values)
        count = int(round(pct * total / 100.0))
        return f"{count} ({pct:.1f}%)"
    return autopct

# Create side-by-side bar and pie charts
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Bar chart
bars = ax1.bar(labels, values, color=colors, edgecolor='black')
ax1.set_ylim(0, 25000)
# ax1.set_xlabel('Outcome')
ax1.set_ylabel('Count')
ax1.set_title('Distribution of Judge Preferences (Bar)')
for bar, value in zip(bars, values):
    pct = value / sum(values) * 100
    ax1.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 500,
        f"{value} ({pct:.1f}%)",
        ha='center',
        va='bottom'
    )

# Pie chart
wedges, texts, autotexts = ax2.pie(
    values,
    colors=colors,
    autopct=make_autopct(values),
    startangle=90,
    pctdistance=0.7,
    textprops={'color': 'black', 'weight': 'normal'}
)
ax2.set_title('Distribution of Judge Preferences (Pie)')
ax2.axis('equal')

# Legend for pie chart
ax2.legend(wedges, labels, title="Outcome", loc='lower left', bbox_to_anchor=(1, 0.5))

# Add a caption centered below the axes
fig.text(
    0.5,
    # x-position in figure coordinates (0-1)

```

```

-0.02,          # y-position in figure coordinates (0-1)
'Figure 1: Distribution of Response Preferences',
ha='center',
va='bottom',
fontsize='medium',
color='black'
)

plt.tight_layout()
plt.show()

# Compute usage counts
usage_counts = df['model_a'].value_counts().add(df['model_b'].value_counts(),
↳fill_value=0)

# Compute preference counts
preference_counts = {}
for _, row in df.iterrows():
    if row['winner_model_a'] == 1:
        preference_counts[row['model_a']] = preference_counts.
↳get(row['model_a'], 0) + 1
    elif row['winner_model_b'] == 1:
        preference_counts[row['model_b']] = preference_counts.
↳get(row['model_b'], 0) + 1

preference_series = pd.Series(preference_counts)

# Select top 10 models by usage
top10 = usage_counts.nlargest(20).index.tolist()

# Filter for top10
usage_top10 = usage_counts[top10]
preference_top10 = preference_series.reindex(top10).fillna(0)

# Plot double bar chart
x = np.arange(len(top10))
width = 0.35

fig, ax = plt.subplots(figsize=(12, 6))
bar1 = ax.bar(x - width/2, usage_top10, width, label='Usage Count',
↳color='orchid', edgecolor='black')
bar2 = ax.bar(x + width/2, preference_top10, width, label='Preference Count',
↳color='skyblue', edgecolor='black')

# Labels and title
# ax.set_xlabel('Model')
ax.set_ylabel('Count')

```

```

ax.set_title('Usage and Preference of the Top 20 Models')
ax.set_xticks(x)
ax.set_xticklabels(top10, rotation=45, ha='right')
ax.legend(loc='upper right')

# Add a caption centered below the axes
fig.text(
    0.5,          # x-position in figure coordinates (0-1)
    -0.02,        # y-position in figure coordinates (0-1)
    'Figure 2: Frequency of Usage and Preference of Top 20 Models',
    ha='center',
    va='bottom',
    fontsize='medium',
    color='black'
)

plt.tight_layout()
plt.show()

# Compute prompt lengths (in tokens)
df['prompt_length'] = df['prompt'].str.split().apply(len)

# Plot the distribution of prompt lengths
fig = plt.figure(figsize=(12, 6))
plt.hist(df['prompt_length'], bins=500, edgecolor='black', color='deepskyblue')
plt.xlim(0, 400)
plt.xticks(np.arange(0, 401, 20))
plt.xlabel('Prompt Length (# of Tokens)')
plt.ylabel('Frequency')
plt.title('Distribution of Prompt Lengths')
# plt.grid(True)

# Add a caption centered below the axes
fig.text(
    0.5,          # x-position in figure coordinates (0-1)
    -0.02,        # y-position in figure coordinates (0-1)
    'Figure 3: Histogram of Prompt Length (in Tokens)',
    ha='center',
    va='bottom',
    fontsize='medium',
    color='black'
)

plt.tight_layout()
plt.show()

# Compute token lengths for each response

```



```

df['resp_a_len'] = df['response_a'].str.split().apply(len)
df['resp_b_len'] = df['response_b'].str.split().apply(len)

# Extract lengths of preferred and unpreferred responses (ignore ties)
pref = np.where(df['winner_model_a'] == 1, df['resp_a_len'],
                np.where(df['winner_model_b'] == 1, df['resp_b_len'], np.nan))
unpref = np.where(df['winner_model_a'] == 1, df['resp_b_len'],
                  np.where(df['winner_model_b'] == 1, df['resp_a_len'], np.nan))

preferred_lengths = pd.Series(pref).dropna().astype(int)
unpreferred_lengths = pd.Series(unpref).dropna().astype(int)

# Plot side-by-side histograms
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6), sharey=True)

bins = 50
x_limits = (0, 1000)
xticks = np.arange(0, 1001, 100)

# Preferred responses
ax1.hist(preferred_lengths, bins=bins, range=x_limits, edgecolor='black',
        color='mediumseagreen')
ax1.set_xlim(*x_limits)
ax1.set_xticks(xticks)
ax1.set_xlabel('Preferred Response Length (# tokens)')
ax1.set_ylabel('Frequency')
ax1.set_title('Preferred Responses')

# Unpreferred responses
ax2.hist(unpreferred_lengths, bins=bins, range=x_limits, edgecolor='black',
        color='salmon')
ax2.set_xlim(*x_limits)
ax2.set_xticks(xticks)
ax2.set_xlabel('Unpreferred Response Length (# tokens)')
ax2.set_title('Unpreferred Responses')

# Caption
fig.text(
    0.5, 0.02,
    'Figure 4: Distribution of Response Lengths for Preferred (Left) vs.
    Unpreferred (Right) Responses (excluding ties)',
    ha='center', va='bottom', fontsize='medium', color='black'
)

# plt.suptitle('Response Length Distribution: Preferred vs. Unpreferred', y=0.92)
plt.tight_layout(rect=[0, 0.05, 1, 0.95])
plt.show()

```

```

# Syllable counting helper
def count_syllables(word):
    word = re.sub(r'^a-zA-Z', '', word).lower()
    syllables = len(re.findall(r'[aeiouy]+', word))
    if word.endswith('e'):
        syllables = max(1, syllables - 1)
    return syllables or 1

# Flesch-Kincaid grade computation
def fk_grade(text):
    sentences = [s for s in re.split(r'[.!?]+', text) if s.strip()]
    num_sent = max(len(sentences), 1)
    words = re.findall(r'\b\w+\b', text)
    num_words = max(len(words), 1)
    syllables = sum(count_syllables(w) for w in words)
    return 0.39 * (num_words / num_sent) + 11.8 * (syllables / num_words) - 15.59

# Compute readability for both responses
df['fk_a'] = df['response_a'].apply(fk_grade)
df['fk_b'] = df['response_b'].apply(fk_grade)

# Gather readability scores for preferred and unpreferred responses (exclude
→ties)
pref_scores = []
unpref_scores = []
for _, row in df.iterrows():
    if row['winner_model_a'] == 1:
        pref_scores.append(row['fk_a'])
        unpref_scores.append(row['fk_b'])
    elif row['winner_model_b'] == 1:
        pref_scores.append(row['fk_b'])
        unpref_scores.append(row['fk_a'])

# Convert to Series
pref_series = pd.Series(pref_scores, name='Preferred')
unpref_series = pd.Series(unpref_scores, name='Unpreferred')

# Optionally cap FK grades to remove extreme outliers
cap = 30
pref_series = pref_series.clip(upper=cap)
unpref_series = unpref_series.clip(upper=cap)

# Plot side-by-side histograms
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6), sharey=True)

bins = 30
ax1.hist(pref_series, bins=bins, edgecolor='black', color='mediumseagreen')

```

```

ax1.set_title('Preferred Responses')
ax1.set_xlabel('Flesch-Kincaid Grade (capped at 30)')
ax1.set_ylabel('Frequency')

ax2.hist(unpref_series, bins=bins, edgecolor='black', color='salmon')
ax2.set_title('Unpreferred Responses')
ax2.set_xlabel('Flesch-Kincaid Grade (capped at 30)')

# plt.suptitle('Readability Distribution: Preferred vs Unpreferred Responses')
# Caption
fig.text(
    0.5, 0,
    'Figure 5: Distribution of Readability Scores for Preferred (Left) vs. Unpreferred (Right) Responses',
    ha='center', va='bottom', fontsize='medium', color='black'
)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

# Define NLTK-based cosine similarity using wordpunct_tokenize
def cosine_sim_nltk(text1, text2):
    tokens1 = wordpunct_tokenize(text1.lower())
    tokens2 = wordpunct_tokenize(text2.lower())
    vec1 = Counter(tokens1)
    vec2 = Counter(tokens2)
    # dot product
    intersection = set(vec1.keys()) & set(vec2.keys())
    numerator = sum(vec1[t] * vec2[t] for t in intersection)
    # norms
    sum1 = sum(v**2 for v in vec1.values())
    sum2 = sum(v**2 for v in vec2.values())
    denominator = math.sqrt(sum1) * math.sqrt(sum2)
    return numerator / denominator if denominator else 0.0

# Compute similarities for each row
df['sim_a_nltk'] = df.apply(lambda x: cosine_sim_nltk(x['prompt'], x['response_a']), axis=1)
df['sim_b_nltk'] = df.apply(lambda x: cosine_sim_nltk(x['prompt'], x['response_b']), axis=1)

# Group preferred vs. unpreferred similarities
pref_sims = np.concatenate([
    df.loc[df['winner_model_a'] == 1, 'sim_a_nltk'].values,
    df.loc[df['winner_model_b'] == 1, 'sim_b_nltk'].values
])
unpref_sims = np.concatenate([

```

```

    df.loc[df['winner_model_a'] == 1, 'sim_b_nltk'].values,
    df.loc[df['winner_model_b'] == 1, 'sim_a_nltk'].values
])

# Plot histograms
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
ax1.hist(pref_sims, bins = 25, range=(0,1), color='mediumseagreen',
        edgecolor='black')
ax1.set_title('Preferred Responses')
ax1.set_xlabel('Cosine Similarity to Prompt')
ax2.hist(unpref_sims, bins = 25, range=(0,1), color='salmon', edgecolor='black')
ax2.set_title('Unpreferred Responses')
ax2.set_xlabel('Cosine Similarity to Prompt')
ax1.set_ylabel('Frequency')

# Caption
fig.text(
    0.5, -0.03,
    'Figure 6: Distribution of Cosine Similarity with the Prompt for Preferred_
    (Left) vs. Unpreferred (Right) Responses',
    ha='center', va='bottom', fontsize='medium', color='black'
)

# plt.suptitle('Prompt-Response Cosine Similarity (NLTK Tokenization)')
plt.tight_layout()
plt.show()

# Split preferred and unpreferred texts
preferred = pd.concat([
    df.loc[df['winner_model_a'] == 1, 'response_a'],
    df.loc[df['winner_model_b'] == 1, 'response_b']
])
unpreferred = pd.concat([
    df.loc[df['winner_model_a'] == 1, 'response_b'],
    df.loc[df['winner_model_b'] == 1, 'response_a']
])

# Preprocess function
stop_words = set(stopwords.words('english'))
def preprocess(series):
    text = series.str.cat(sep=' ').lower()
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t.isalpha() and t not in stop_words]
    return ' '.join(tokens)

# Preprocess the text
pref_text = preprocess(preferred)

```

```

unpref_text = preprocess(unpreferred)

# Generate word clouds
wc_pref = WordCloud(width=400, height=400, background_color='white').
    ↳generate(pref_text)
wc_unpref = WordCloud(width=400, height=400, background_color='white').
    ↳generate(unpref_text)

# Create the plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))
ax1.imshow(wc_pref, interpolation='bilinear')
ax1.axis('off')
ax1.set_title('Preferred Responses')
ax2.imshow(wc_unpref, interpolation='bilinear')
ax2.axis('off')
ax2.set_title('Unpreferred Responses')

# Caption
fig.text(
    0.5, -0.03,
    'Figure 7: Most Common Words for Preferred (Left) vs. Unpreferred (Right)↳
    ↳Responses',
    ha='center', va='bottom', fontsize='medium', color='black'
)

plt.tight_layout()
plt.show()

```

```

[ ]: # Load the training and testing datasets
train_df, test_df = df, pd.read_csv('test.csv', index_col='id')

# Perform Feature Engineering for Training Dataset
train_df['resp_len_diff'] = train_df['resp_a_len'] - train_df['resp_b_len']
train_df['fk_diff']      = train_df['fk_a'] - train_df['fk_b']
train_df['sim_diff']     = train_df['sim_a_nltk'] - train_df['sim_b_nltk']

# Perform Feature Engineering on Test Dataset
test_df['prompt_length'] = test_df['prompt'].str.split().apply(len)
test_df['resp_a_len']    = test_df['response_a'].str.split().apply(len)
test_df['resp_b_len']    = test_df['response_b'].str.split().apply(len)
test_df['fk_a']          = test_df['response_a'].apply(fk_grade)
test_df['fk_b']          = test_df['response_b'].apply(fk_grade)
test_df['sim_a_nltk']    = test_df.apply(lambda r: cosine_sim_nltk(r['prompt'],↳
    ↳r['response_a']), axis=1)
test_df['sim_b_nltk']    = test_df.apply(lambda r: cosine_sim_nltk(r['prompt'],↳
    ↳r['response_b']), axis=1)
test_df['resp_len_diff'] = test_df['resp_a_len'] - test_df['resp_b_len']

```

```

test_df['fk_diff']          = test_df['fk_a'] - test_df['fk_b']
test_df['sim_diff']         = test_df['sim_a_nltk'] - test_df['sim_b_nltk']

# ===== Logistic Regression =====
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

features = ['prompt_length', 'resp_len_diff', 'fk_diff', 'sim_diff']

submission_lr = pd.DataFrame(index=test_df.index)

for col in ['winner_model_a', 'winner_model_b', 'winner_tie']:
    y = train_df[col].values
    X_tr, X_cv, y_tr, y_cv = train_test_split(
        train_df[features], y,
        stratify=y, test_size=0.2, random_state=42
    )
    gs = GridSearchCV(
        LogisticRegression(solver='lbfgs', max_iter=1000),
        {'C': [0.01, 0.1, 1, 10]},
        scoring='accuracy', cv=3, n_jobs=-1
    )
    gs.fit(X_tr, y_tr)
    best = gs.best_estimator_
    print(f"{col} LR best C:", gs.best_params_['C'])
    print(f"{col} LR CV accuracy:", accuracy_score(y_cv, best.predict(X_cv)))
    submission_lr[col] = best.predict_proba(test_df[features])[:, 1]

submission_lr.to_csv('submission_LR.csv', index_label='id')

# ===== Naive Bayes =====
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

features = ['prompt_length', 'resp_len_diff', 'fk_diff', 'sim_diff']

submission_nb = pd.DataFrame(index=test_df.index)

for col in ['winner_model_a', 'winner_model_b', 'winner_tie']:
    y = train_df[col].values
    X_tr, X_cv, y_tr, y_cv = train_test_split(
        train_df[features], y,
        stratify=y, test_size=0.2, random_state=42
    )

```

```

)
gs = GridSearchCV(
    GaussianNB(),
    {'var_smoothing': [1e-9,1e-8,1e-7]},
    scoring='accuracy', cv=3
)
gs.fit(X_tr, y_tr)
best = gs.best_estimator_
print(f"{col} NB best var_smoothing:", gs.best_params_['var_smoothing'])
print(f"{col} NB CV accuracy:", accuracy_score(y_cv, best.predict(X_cv)))
submission_nb[col] = best.predict_proba(test_df[features])[:,1]

submission_nb.to_csv('submission_NB.csv', index_label='id')

# ===== Random Forest =====
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

features = ['prompt_length', 'resp_len_diff', 'fk_diff', 'sim_diff']

submission_rf = pd.DataFrame(index=test_df.index)

for col in ['winner_model_a', 'winner_model_b', 'winner_tie']:
    y = train_df[col].values
    X_tr, X_cv, y_tr, y_cv = train_test_split(
        train_df[features], y,
        stratify=y, test_size=0.2, random_state=42
    )
    gs = GridSearchCV(
        RandomForestClassifier(random_state=42),
        {'n_estimators': [100,200], 'max_depth': [5,10]},
        scoring='accuracy', cv=3, n_jobs=-1
    )
    gs.fit(X_tr, y_tr)
    best = gs.best_estimator_
    print(f"{col} RF best params:", gs.best_params_)
    print(f"{col} RF CV accuracy:", accuracy_score(y_cv, best.predict(X_cv)))
    submission_rf[col] = best.predict_proba(test_df[features])[:,1]

submission_rf.to_csv('submission_RF.csv', index_label='id')

# ===== XGBoost =====
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from xgboost import XGBClassifier

```

```

from sklearn.metrics import accuracy_score

features = ['prompt_length', 'resp_len_diff', 'fk_diff', 'sim_diff']

submission_xgb = pd.DataFrame(index=test_df.index)

for col in ['winner_model_a', 'winner_model_b', 'winner_tie']:
    y = train_df[col].values
    X_tr, X_cv, y_tr, y_cv = train_test_split(
        train_df[features], y,
        stratify=y, test_size=0.2, random_state=42
    )
    gs = GridSearchCV(
        XGBClassifier(use_label_encoder=False, eval_metric='logloss',
        ↪random_state=42),
        {'learning_rate':[0.01,0.1], 'max_depth':[4,6], 'n_estimators':
        ↪[100,200]},
        scoring='accuracy', cv=3, n_jobs=-1
    )
    gs.fit(X_tr, y_tr)
    best = gs.best_estimator_
    print(f"{col} XGB best params:", gs.best_params_)
    print(f"{col} XGB CV accuracy:", accuracy_score(y_cv, best.predict(X_cv)))
    submission_xgb[col] = best.predict_proba(test_df[features])[:,1]

submission_xgb.to_csv('submission_XGB.csv', index_label='id')

# Filter & encode labels
train_df = train_df[train_df[['winner_model_a', 'winner_model_b', 'winner_tie']].
    ↪sum(axis=1) == 1]
label_map = {'winner_model_a': 0, 'winner_model_b': 1, 'winner_tie': 2}
train_df['label'] = train_df[['winner_model_a', 'winner_model_b', 'winner_tie']].
    ↪idxmax(axis=1).map(label_map)

# Build model inputs
def build_input(row): return f"{row['prompt']} [SEP] {row['response_a']} [SEP]
    ↪{row['response_b']}"
texts = train_df.apply(build_input, axis=1).tolist()
labels = train_df['label'].tolist()
test_texts = test_df.apply(build_input, axis=1).tolist()

# Split for validation
train_texts, val_texts, train_labels, val_labels = train_test_split(
    texts, labels, test_size=0.1, stratify=labels, random_state=88
)

# Create tokenizer and dataset

```



```

model_name = "roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)

class PreferenceDataset(Dataset):
    def __init__(self, texts, labels=None, tokenizer=None, max_len=256):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len
    def __len__(self):
        return len(self.texts)
    def __getitem__(self, idx):
        enc = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=self.max_len,
            return_tensors='pt'
        )
        item = {k: v.squeeze(0) for k,v in enc.items()}
        if self.labels is not None:
            item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

train_ds = PreferenceDataset(train_texts, train_labels, tokenizer)
val_ds   = PreferenceDataset(val_texts,  val_labels,  tokenizer)
test_ds  = PreferenceDataset(test_texts,  labels=None, tokenizer=tokenizer)

# Load model
model = AutoModelForSequenceClassification.from_pretrained(model_name,
    ↪num_labels=3)

# Set training arguments and compute metrics
args = TrainingArguments(
    output_dir="transformer_out",
    #     evaluation_strategy="epoch",
    save_strategy="no",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    learning_rate=2e-5,
    weight_decay=0.01,
    logging_steps=50,
    seed=42
)

def compute_metrics(eval_pred):
    logits, labels = eval_pred

```

```

    preds = np.argmax(logits, axis=1)
    acc = (preds == labels).mean()
    return {"accuracy": acc}

# Train the model
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics,
    tokenizer=tokenizer
)
trainer.train()

# Predict on the test data
preds = trainer.predict(test_ds).predictions # shape (n_test, 3)
probs = torch.softmax(torch.tensor(preds), dim=1).numpy()

# Build and save submission
submission = pd.DataFrame(
    probs,
    columns=["winner_model_a", "winner_model_b", "winner_tie"]
)
# if test.csv has an 'id' column use it, else use its index
if "id" in test_df.columns:
    submission.insert(0, "id", test_df["id"])
else:
    submission.insert(0, "id", test_df.index)
os.makedirs("output", exist_ok=True)
submission.to_csv("output/submission_transformer.csv", index=False)
print("Saved output/submission_transformer.csv")

```